Sistemas Operativos 2026/2027 - Trabalho Prático 2

Simulador de substituição de páginas

Autor: José Sá | Número: a22504133

Repositório GitHub: POR PREENCHER ANTES DA ENTREGA.

1. Objetivo

O objetivo foi acrescentar RANDOM, LRU, NRU e CLOCK ao simulador que já tinha FIFO, criar cenários e comparar as faltas de página e as operações de swap. Foi mantido o escalonador FIFO. As tarefas opcionais de RR não foram realizadas.

2. Funcionamento

Uma falta de página acontece quando a página pedida não está na RAM. Se for preciso libertar uma frame, o algoritmo de substituição escolhe a página a remover.

• FIFO remove a página carregada há mais tempo.

• RANDOM escolhe aleatoriamente uma das frames ocupadas.

• LRU remove a página cujo último acesso é mais antigo.

• NRU escolhe primeiro uma página com R=0. Nesta implementação usa um bit de referência, limpo antes dos acessos 1, 5, 9, 13, etc. Se todas tiverem R=1, escolhe a mais antiga. Quando há várias com R=0, o desempate também é pela ordem de carregamento.

• CLOCK percorre circularmente as páginas. Se encontra R=1, limpa esse bit e avança. Remove a primeira página com R=0. A pesquisa seguinte começa no sucessor da vítima.

O NRU usado é a variante de um bit; não utiliza classes que combinem R e dirty. No LRU, last_accessed é um instante lógico: aumenta uma unidade por acesso, mesmo quando vários acessos acontecem no mesmo milissegundo.

Uma leitura não torna uma página nova suja. Uma escrita, indicada por um número negativo no CSV, coloca dirty=1. Expulsar uma página suja gera um swap-out. Voltar a carregar uma página com cópia em swap gera um swap-in. Essa cópia é mantida, e a página carregada fica limpa até ser escrita novamente. Uma página limpa sem cópia em swap pode ser descartada e criada novamente. O conteúdo dos dados e os tempos de disco não são simulados: contam-se as operações.

Foram corrigidos a indexação das páginas (1 até vpages), o uso efetivo de --vpages, a libertação das frames e a substituição antes de faltas, evitando expulsões em HITs. O ficheiro básico tinha uma página 0, que foi removida por não ser válida no formato. O código fornecido da aplicação e do leitor de CSV não foi alterado.

3. Configuração e cenários

As simulações foram executadas em Linux (imagem DOLOS), com GCC 14.2.0, uma aplicação, escalonador FIFO, 8 páginas virtuais, 3 frames e threshold=1. Antes de carregar uma página em falta, o simulador garante pelo menos threshold frames livres. Com threshold=1 pode utilizar as três frames.

Cada linha dos quatro cenários contém um burst de 10 ms, sem bloqueio, nice=0 e uma referência a uma página. Abaixo, valores negativos indicam escritas.

• repeticao: 1,2,3,1,2,3,1,2,3. As três páginas cabem na RAM.

• localidade: 1,2,3,1,4,1,5,1,2,1,3,1,4,1,5,1. A página 1 é reutilizada frequentemente.

• escritas: -1,-2,3,4,1,2,-1,5,1,2,3,4. Obriga a expulsar páginas sujas e a recarregá-las.

• circular: 1,2,3,4,5,1,2,3,4,5. Há mais páginas utilizadas do que frames disponíveis.

4. Resultados dos algoritmos deterministas

Os valores foram recolhidos das linhas stats dos logs. Swap-in corresponde a leituras de swap; swap-out corresponde a escritas em swap.

Cenário	Algoritmo	Acessos	Faltas	Swap-in	Swap-out
repeticao	FIFO	9	3	0	0
repeticao	LRU	9	3	0	0
repeticao	NRU	9	3	0	0
repeticao	CLOCK	9	3	0	0
localidade	FIFO	16	11	0	0
localidade	LRU	16	9	0	0
localidade	NRU	16	11	0	0
localidade	CLOCK	16	10	0	0
escritas	FIFO	12	9	2	3
escritas	LRU	12	9	2	3
escritas	NRU	12	9	2	3
escritas	CLOCK	12	9	2	3
circular	FIFO	10	10	0	0
circular	LRU	10	10	0	0
circular	NRU	10	10	0	0
circular	CLOCK	10	10	0	0

No cenário repeticao, todos têm apenas as três faltas iniciais. No localidade, o LRU conserva melhor a página usada com frequência e tem 9 faltas, contra 11 do FIFO e 10 do CLOCK. O NRU teve 11 faltas nesta configuração de limpeza dos bits.

No cenário escritas, os quatro tiveram os mesmos totais: 9 faltas, 2 leituras e 3 escritas de swap. As restantes faltas não são leituras de swap, porque incluem páginas novas e páginas limpas que foram descartadas. No circular, todos tiveram 10 faltas: as páginas voltam a ser pedidas depois de terem sido removidas.

5. Resultados do RANDOM

Foram usadas as seeds 1, 7 e 42, definidas por OSSIM_SEED. A mesma seed reproduz as escolhas no mesmo ambiente; uma biblioteca C diferente pode gerar outra sequência.

Cenário	Algoritmo	Seed	Acessos	Faltas	Swap-in	Swap-out
repeticao	RANDOM	1	9	3	0	0
repeticao	RANDOM	7	9	3	0	0
repeticao	RANDOM	42	9	3	0	0
localidade	RANDOM	1	16	10	0	0
localidade	RANDOM	7	16	14	0	0
localidade	RANDOM	42	16	11	0	0
escritas	RANDOM	1	12	9	3	2
escritas	RANDOM	7	12	11	4	3
escritas	RANDOM	42	12	10	4	3
circular	RANDOM	1	10	8	0	0
circular	RANDOM	7	10	9	0	0
circular	RANDOM	42	10	9	0	0

No cenário localidade, o RANDOM variou entre 10 e 14 faltas. No circular obteve 8 ou 9 faltas, menos do que os 10 dos outros algoritmos. Isto mostra que o resultado depende da sequência e das escolhas aleatórias; não prova que o RANDOM seja melhor em geral. No cenário escritas também variaram as leituras e as escritas em swap.

6. Verificação e reprodução

O programa testar.py executa 28 simulações e guarda os logs e resumo.csv. Para FIFO, LRU, NRU e CLOCK compara a sequência completa de vítimas e os totais com um modelo independente em Python. Para RANDOM verifica os limites das contagens e a repetição com seed fixa. Também testa argumentos inválidos.

O teste tests/memoria.c foi executado com AddressSanitizer e UndefinedBehaviorSanitizer. Verificou limites das páginas, ausência de frames livres duplicadas, correspondência entre frames ocupadas e a ordem de substituição, e libertação com dois processos. Não foram detetados erros nesses testes. Os testes não constituem uma prova para todos os inputs possíveis.

Para repetir: cmake -S . -B build; cmake --build build; python3 testar.py. Para uma demonstração: bash run.sh --vm LRU --vpages 8 --pframes 3 --threshold 1 --scen cenarios/localidade.csv. Os logs indicam HIT/FAULT, a vítima escolhida, as operações de swap, as segundas oportunidades do CLOCK e a limpeza do NRU.

7. Conclusão

O LRU teve o melhor resultado determinista no cenário de localidade. Nos restantes, houve resultados iguais entre os algoritmos deterministas. O RANDOM variou com a seed. Não há um vencedor universal nestes testes. As faltas e o custo de swap devem ser analisados separadamente, porque uma falta nem sempre exige uma leitura de disco e uma expulsão limpa não exige uma escrita.

Antes da entrega falta identificar o repositório GitHub e convidar os professores como participantes. O ZIP reúne o código, os cenários, os resultados e este relatório.